Project- Low Level Design
On
MedScribe AI

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Vinay Kuthey	EN22CS3011084
2	Viraj Agrawal	EN22CS3011089
3	Lakshita Singh	EN22CS3011135
4	Vishakha Shadija	EN22ME304114
5	Vishakha Hilsayan	EN22EL301065

Group Name: 09D10

Project Number: GAI-22

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2026

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. **References**

1. Introduction

1.1 Scope of the document

This document describes the low-level design of the MedScribe AI. It covers internal architecture, module-level design, data flow, API structure, and implementation details required to build and maintain the system.

1.2 Intended Audience

- Developers
- Testers
- Project evaluators
- Technical reviewers

1.3 System Overview

The Healthcare Content Generator is a GenAI-based application designed to generate structured medical documents such as patient summaries. The system uses Retrieval-Augmented Generation (RAG) by combining a vector database (ChromaDB) with a Large Language Model (Gemini API). It ensures consistent formatting, professional tone, and domain-specific terminology.

2. System Design

2.1 Application Design

The system follows a layered architecture consisting of three main layers: Frontend Layer, Backend Layer, and AI & Data Layer. It implements a Retrieval-Augmented Generation (RAG) pipeline to generate accurate and structured healthcare content.

Frontend Layer (User Interface)

- Built using HTML, CSS, and JavaScript
- Allows users (healthcare professionals) to input patient details
- Sends data to backend via HTTP POST (/generate)
- Displays the generated patient summary

Backend Layer (Flask)

- Acts as the core processing unit
- Handles API requests through the API Controller (app.py)
- Manages the RAG pipeline workflow
- Coordinates between embedding model, vector database, and LLM
- Uses a Prompt Engineering Module to:
    - Structure output in SOAP format
    - Maintain medical terminology and consistency

AI & Data Layer

This layer provides intelligence and context-aware generation:

- **Embedding Model (SentenceTransformer):**
    - Converts input text into vector embeddings
- **Vector Database (ChromaDB):**
    - Stores embeddings and generated documents
    - Retrieves similar past records for context (RAG)
- **LLM API (Gemini 2.5 Flash):**
    - Generates structured patient summaries
    - Uses both user input and retrieved context

Architecture:

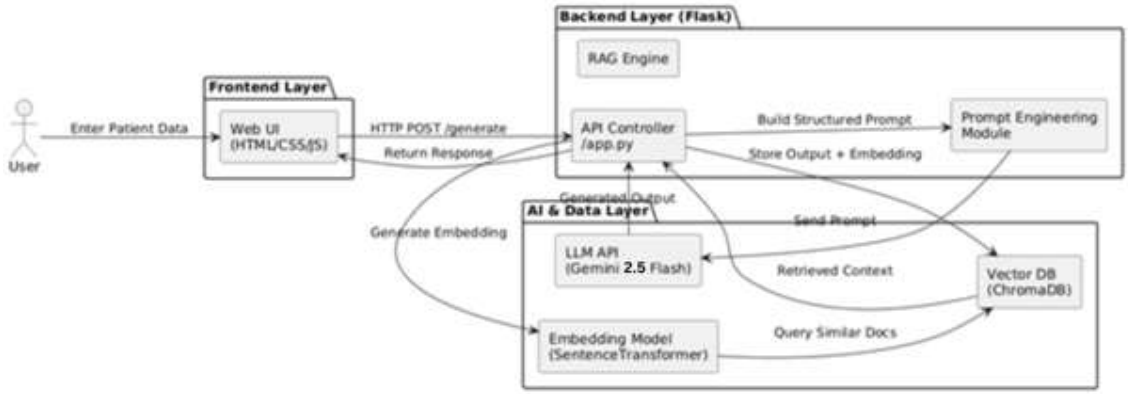
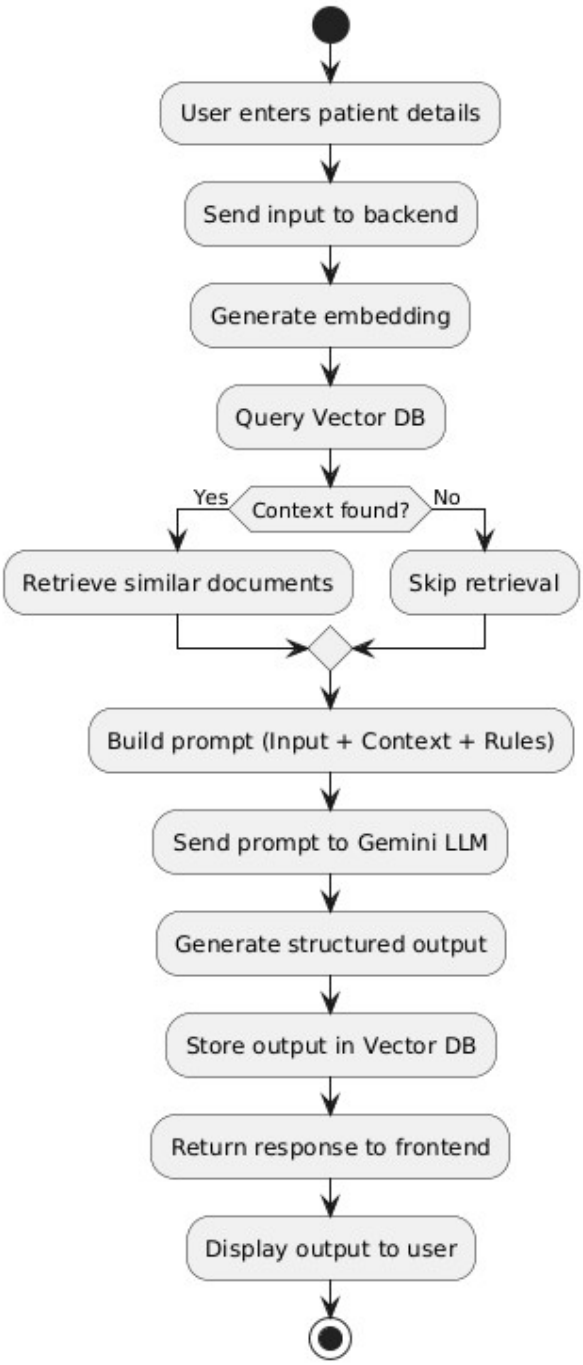


Fig. Architecture Diagram

2.2 Process Flow

Activity Diagram:



Fig, Activity Diagram

Workflow Description:

User Input:
The process starts when the user enters patient details in the frontend.

Send to Backend:
The input is sent to the Flask backend via an API request.

Embedding Generation:
The backend converts the input text into vector embeddings using the embedding model.

Vector DB Query:
The system queries the vector database to find similar past documents.

Decision – Context Found?:

- **Yes:** Relevant documents are retrieved as context
- **No:** Retrieval step is skipped

Prompt Construction:
A structured prompt is created using:

- User input
- Retrieved context (if available)
- Predefined rules (SOAP format, medical terminology)

LLM Processing:
The prompt is sent to the Gemini LLM for content generation.

Output Generation:
The LLM generates a structured patient summary.

Store Output:
The generated output is stored in the vector database for future retrieval.

Return Response:
The backend sends the response back to the frontend.

Display Output:
The final output is displayed to the user.

2.3 Information Flow

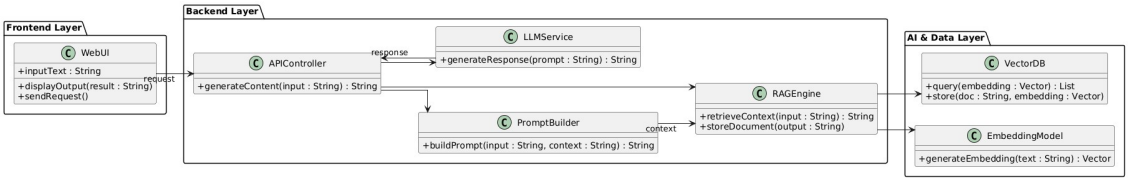
- Input Data: Patient case description
- Processed Data: Embeddings (vector format)
- Context Data: Retrieved similar documents
- Output Data: Structured patient summary (SOAP format)

Flow:

User Input → Embedding → Vector DB → Context → Prompt → LLM → Output → Storage

2.4 Components Design

Class Diagram:



Relationships Between Classes

- **WebUI → APIController:** sends user request
- **APIController → RAGEngine:** handles retrieval & storage
- **APIController → PromptBuilder:** constructs prompt
- **APIController → LLMService:** generates output
- **RAGEngine → VectorDB & EmbeddingModel:** for retrieval and storage

2.5 Key Design Considerations

Accuracy: Use RAG to improve contextual relevance

Consistency: Enforce structured format (SOAP)

Scalability: Modular backend design

Performance: Limit retrieval results (top-k)

Security: Avoid sensitive data exposure

Maintainability: Clean separation of modules

2.6 API Catalogue

Generate Content API

- Endpoint: /generate
- Method: POST

Request Body: Input via form

Response: Generated Summary

3. References

- Gemini API Documentation
- ChromaDB Documentation
- Sentence Transformers Documentation
- Flask Official Documentation
- Retrieval-Augmented Generation (RAG) Concepts